

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: 30 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Se evidencia al demostrar que un desarrollador profesional no entrega código sin haberlo probado. Cada prueba escrita es un compromiso con el equipo y con el cliente de que el código hace lo que dice hacer. La cobertura de código no es una métrica de decoración: es evidencia de responsabilidad técnica.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Implementa pruebas unitarias para validar componentes de software, asegurando que el código funciona correctamente de forma aislada y frente a sus dependencias.
Competencia Específica	Escribe pruebas unitarias usando Jest (Node.js) o pytest (Python), aplica mocks, stubs y fakes para aislar dependencias externas, ejecuta el reporte de cobertura e interpreta los resultados para identificar código sin testear.

2. Palabras Clave

- **Prueba unitaria:** Prueba que verifica el comportamiento de una unidad de código (función o clase) de forma aislada de sus dependencias.
- **Mock:** Objeto simulado que reemplaza una dependencia real y permite verificar que fue llamado con los parámetros correctos.
- **Stub:** Versión simplificada de una dependencia que retorna valores predefinidos sin ejecutar lógica real.
- **Fake:** Implementación funcional pero simplificada de una dependencia, usada en pruebas cuando un stub es insuficiente.

- **Cobertura de código:** Porcentaje de líneas, ramas o funciones del código que son ejecutadas por las pruebas escritas.

3. Resumen

Este ejercicio demostrativo presenta el ciclo completo de pruebas unitarias: partiendo de un fragmento de código real con dependencias externas (autenticación y agenda de citas), se escriben pruebas unitarias usando Jest en Node.js, se aplican mocks, stubs y fakes para aislar las dependencias, se ejecutan las pruebas y se revisa el reporte de cobertura para identificar código no testeado. Como cierre, se muestra una demostración rápida de refactorización asistida con IA. El objetivo es que el estudiante comprenda el ciclo completo: código limpio → prueba → ejecución → cobertura.-----4.

4. Contenido

Descripción del ejercicio

Se trabaja sobre una API de gestión de citas médicas con dos módulos: `authService.js` (autenticación de usuarios) y `appointmentService.js` (gestión de citas). Ambos tienen dependencias externas como base de datos y servicio de correo. El ejercicio demuestra cómo probar estos módulos de forma aislada sin necesidad de una base de datos real.

Objetivo de la demostración

Demostrar el proceso completo de escritura, ejecución e interpretación de pruebas unitarias con aislamiento de dependencias, con el fin de que el estudiante comprenda que escribir código y escribir pruebas son responsabilidades inseparables del desarrollo profesional.

Recursos necesarios

- Computadora con Node.js y npm instalados (versión LTS).
- Editor VS Code con extensión Jest Runner.
- Repositorio base del ejercicio clonado previamente.
- Terminal integrada en VS Code.
- Proyector o pantalla compartida para que todos vean la ejecución en tiempo real.

Desarrollo paso a paso

Paso 1 — Recorrido del código a testear (15 min)

El auxiliar presenta los tres archivos del proyecto y explica su estructura:

- `src/authService.js`: Contiene la función `login(email, password)` que consulta la base de datos y retorna un token JWT. Depende de un módulo de base de datos (`db`) y de un generador de tokens (`tokenService`).
- `src/appointmentService.js`: Contiene `createAppointment(patientId, doctorId, date)` que guarda una cita y envía un correo de confirmación. Depende de `db` y del servicio de correo (`emailService`).
- `src/app.js`: Archivo principal que conecta ambos servicios con las rutas de la API. El estudiante debe observar que si se prueba `login()` directamente, se necesitaría una base de datos real con datos de prueba, lo que hace la prueba lenta, frágil y dependiente del entorno.

Paso 2 — Estructura de una prueba unitaria (5 min)

El auxiliar muestra la anatomía de una prueba con Jest, explicando el patrón **Arrange-Act-Assert**.

JavaScript

```
describe('authService', () => {  
  it('debe retornar un token cuando las credenciales son válidas',  
    async () => {  
      // Arrange: preparar datos y mocks  
      // Act: ejecutar la función a probar  
      // Assert: verificar el resultado  
    });  
});
```

Paso 3 — Aplicar Mock, Stub y Fake (15 min)

El auxiliar demuestra cada concepto con el código del proyecto:

- **Mock** — Verificar que una función fue llamada:

JavaScript

```
const tokenService = { generate:
jest.fn().mockReturnValue('token-123') };

// Después de ejecutar login():
expect(tokenService.generate).toHaveBeenCalled(user.id);
```

- **Stub** — Retornar un valor predefinido sin lógica real:

JavaScript

```
const db = { findUser: jest.fn().mockResolvedValue({ id: 1, email:
'test@test.com' }) };;
```

- **Fake** — Implementación simplificada pero funcional:

JavaScript

```
const fakeEmailService = {
  sent: [],
  send: async (to, subject, body) => { fakeEmailService.sent.push({
to, subject }); }
};
```

Paso 4 — Ejecutar pruebas y revisar resultados (10 min)

El auxiliar ejecuta las pruebas en terminal: `npm test`.

Se muestra la salida de Jest con pruebas pasadas y falladas. Luego se demuestra un fallo intencional modificando el valor esperado para que el estudiante entienda cómo leer los mensajes de error.

Paso 5 — Revisar cobertura de código (10 min)

Se ejecuta el reporte de cobertura: `npm test -- --coverage`.

El auxiliar abre el reporte HTML generado en `coverage/lcov-report/index.html` y explica cómo interpretar:

- **Líneas verdes:** cubiertas por las pruebas.
- **Líneas rojas:** no cubiertas; representan riesgo de bugs no detectados.
- **Ramas (Branches):** condiciones `if/else` que deben probarse en ambos caminos.

Se identifica una rama no cubierta y se escribe la prueba faltante en tiempo real.

Paso 6 — Bonus: Refactorización con IA (5 min)

Se muestra cómo usar la IA integrada en VS Code (GitHub Copilot o equivalente) para sugerir mejoras en el código o en las pruebas, haciendo énfasis en que la IA asiste pero el desarrollador valida. Observaciones importantes

- Un mock no reemplaza una prueba de integración: sirve para probar la unidad en aislamiento; las pruebas de integración verifican que las partes reales funcionan juntas.
- La cobertura del 100% no garantiza que el código sea correcto; garantiza que fue ejecutado. La calidad de los asserts importa tanto como la cobertura.
- Nombrar las pruebas con oraciones completas ("debe retornar error cuando la contraseña es incorrecta") facilita entender qué falló sin leer el código de la prueba.
- El repositorio base debe estar clonado y con `npm install` ejecutado antes de la sesión para no perder tiempo en instalaciones.

Tabla 1 — Diferencia entre Mock, Stub y Fake

Concepto	¿Qué hace?	¿Cuándo usarlo?	Ejemplo en el ejercicio
Mock	Simula una dependencia y verifica cómo fue llamada	Cuando importa <i>que</i> se llamó la función y <i>con qué</i> parámetros	Verificar que <code>tokenService.generate</code> fue llamado con el ID correcto
Stub	Retorna un valor predefinido sin lógica	Cuando solo se necesita controlar qué retorna la dependencia	<code>db.findUser</code> que retorna un usuario de prueba
Fake	Implementación funcional simplificada	Cuando el stub no es suficiente y se necesita algo de comportamiento real	<code>fakeEmailService</code> que registra los correos enviados sin conectarse a SMTP

Resultado esperado

Al finalizar la demostración, el estudiante debe ser capaz de escribir una prueba unitaria básica con Jest, aplicar mocks y stubs para aislar dependencias externas, ejecutar las pruebas desde terminal y leer el reporte de cobertura para identificar qué partes del código no han sido testeadas.

Aplicación práctica

Las pruebas unitarias son un requisito directo del proyecto del curso y del Corto correspondiente a calidad de software. Comprender la diferencia entre mock, stub y fake evita uno de los errores más frecuentes en los proyectos: escribir pruebas que en realidad se conectan a servicios reales y fallan en entornos donde esos servicios no están disponibles.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- **Dificultades encontradas:** Los estudiantes confunden mock y stub con frecuencia, especialmente porque Jest permite usar `jest.fn()` para ambos propósitos. También cuesta entender por qué una prueba que "pasa" puede no estar probando lo que se cree si los asserts son incorrectos o insuficientes.
- **Recomendaciones para futuras sesiones:** Dedicar al menos 5 minutos extras a mostrar una prueba mal escrita (que pasa aunque el código esté roto) para que los estudiantes entiendan que el assert es tan importante como el mock. Para el reporte de cobertura, mostrar primero una función sin pruebas y luego agregar la prueba para que vean el cambio en tiempo real.
- **Qué puntos consideras fue de mayor dificultad para el estudiante:** Interpretar el reporte de cobertura de ramas (branches), ya que requiere entender que cada `if` tiene dos caminos posibles. También la configuración inicial del entorno puede consumir tiempo si los estudiantes no tienen Node.js instalado.

6. Referencias

1. Jest. (2024). *Jest Documentation*. <https://jestjs.io/docs/getting-started>
2. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
3. Oshero, R. (2013). *The Art of Unit Testing* (2nd ed.). Manning Publications.